

OCR Word Search Solver Project

Defense Report 1

Helios Bringuet

Nina Justo

Emile Lassalle

Martin Doillon

EPITA A1

October 31, 2025

Contents

1	Introduction	3
2	Context and subject overview	3
3	Roles and missions of each member	4
4	Objectives, task planning, and project progress	5
5	Challenges, technological and algorithmic choices	7
6	Conclusion	16

1. Introduction

This report presents the conception and development of the **OCR Word Search Solver Project**, a computer vision application capable of automatically reading, interpreting, and solving word search puzzles from images.

The goal of this project is to combine multiple computer science domains — image processing, segmentation, neural networks, and algorithmic problem solving — into a cohesive and functional application.

2. Context and subject overview

The project's objective is to build a complete OCR (Optical Character Recognition) software capable of solving a word search puzzle. Given an image of a printed grid, the program should:

- Load the image;
- Remove colors and convert to black and white;
- Perform pretreatment and noise reduction;
- Locate the grid and the list of words;
- Segment the image into smaller parts — cells, letters, and words;
- Recognize characters from images using a trained neural network;
- Reconstruct the grid and the word list as textual arrays;
- Solve the puzzle algorithmically;
- Display and optionally save the final solved grid.

The final version will include a graphical interface allowing users to visualize, correct, and solve grids interactively.

3. Roles and missions of each member

Helios Bringuet – Neural Network:

Responsible for designing and implementing a neural network capable of recognizing letters extracted from the puzzle image.

Nina Justo – Image Treatment:

Focused on loading, cleaning, and preprocessing the input images, including color removal and rotation.

Emile Lassalle – Solver Algorithm:

Developed the algorithm that finds all listed words within the reconstructed grid.

Martin Doillon – Image Segmentation:

Handled detection of key regions (grid, word list, cells) and segmentation into sub-images.

Team organization:

Throughout the development of the project, the team organized several in-person meetings each week at EPITA to coordinate progress, discuss implementation details, and review completed work. These meetings usually took place in the late afternoon after classes, allowing everyone to share updates on their assigned modules — such as the neural network, image preprocessing, segmentation, and solver components. We used these sessions not only to synchronize our technical choices but also to debug issues together on the school’s machines, which helped ensure that the entire codebase compiled and ran correctly in a consistent environment. Version control was managed through Git, with each team member working on their own branch to avoid merge conflicts. Branches were dedicated to specific features or tasks (for example, “nn-training”, “img-preprocess”, or “solver-debug”), and were only merged into the main branch after a review by another team member. Regular commits and pull requests made it easy to track the evolution of the project and revert changes when necessary. This workflow fostered

clear collaboration, accountability, and structure, allowing the team to integrate everyone’s contributions efficiently while maintaining a clean and stable codebase throughout the development cycle.

4. Objectives, task planning, and project progress

The main objective of the project is to create a robust OCR solver for word search puzzles, combining image preprocessing, segmentation, neural network-based character recognition, and a dedicated search algorithm to identify and highlight the solution words within a given grid image. The final goal is to provide a functional desktop application that can automatically process an image of a word search, detect the grid and the list of words, recognize the letters using a trained model, and output the fully solved grid.

From the beginning, the team agreed on several measurable performance goals to evaluate the quality of the final system:

- An OCR character recognition accuracy rate of at least 90% on clean test images.
- A total processing time under 5 seconds for a standard 15x15 grid on a mid-range laptop.
- Successful detection and segmentation of the grid and word list for at least 95% of properly scanned puzzles.
- Robust handling of minor rotations, lighting inconsistencies, and variations in contrast.

These targets were designed to guide both the technical choices and the testing phases. Achieving them required careful balancing between performance, model complexity, and reliability of image operations. The system’s modular design also ensured that each component — preprocessing, segmentation, neural recognition, and solving — could be developed and tested independently before integration.

The project was structured into multiple phases, spread across the semester:

-
- **Phase 1 – Research, tool selection, repository setup (Mid–September):** The team explored OCR techniques and image processing tools. SDL2 and SDL2_image were chosen for handling images in C. The Git repository was initialized for version control, and a basic Makefile was created to support compilation on Linux systems.
 - **Phase 2 – Image loading, preprocessing, and color removal (Late September):** The preprocessing module was implemented to load PNG images, convert them to grayscale, enhance contrast, and allow manual rotation to correct misaligned images. This ensures normalized inputs for later stages.
 - **Phase 3 – Grid and word list detection (Early October):** Martin focused on segmenting the puzzle image into the grid area, word list, and individual letter cells using thresholding and simple geometric heuristics. Outputs were designed to be ready for eventual integration with the recognition module.
 - **Phase 4 – Letter segmentation and XNOR network training (Mid–October):** Helios developed a small XNOR network to recognize binary patterns representing letters. The network was trained offline on synthetic datasets, and parameters can be saved and loaded. This serves as a proof-of-concept for neural network integration with image inputs.
 - **Phase 5 – Solver implementation (Late October):** Emile implemented the solver algorithm to operate on textual grids and word lists. Currently, it functions purely with character arrays and does not yet interact with image input or the neural network. It can detect horizontal, vertical, and diagonal words in both directions. Early tests focused on verifying correctness with synthetic text-based grids, preparing the module for future integration.

The project has so far followed the expected timeline established at the beginning of the semester, which included repository setup and task planning

in September, the first defense scheduled for early November, and a final presentation in December. As of now, the team is generally on track.

By the end of October, the image loading, grayscale conversion, and contrast enhancement components were fully functional. The segmentation system, responsible for detecting the grid and the word list within the puzzle, is also producing promising results, though a few edge cases still need refinement. The XNOR neural network and the solver are done.

In terms of collaboration, the team has been working efficiently using a Git-based version control workflow. Each member developed their module on separate branches to prevent conflicts, with merges performed only after successful testing and compilation. Weekly in-person meetings at EPITA have been particularly valuable for debugging and coordinating the integration process, allowing everyone to stay aligned on technical objectives and deadlines.

Overall, the project is progressing steadily and in line with expectations for the first defense. The foundation of the OCR pipeline is in place and the focus for the next few weeks will be on completing the OCR recognition and putting everything together, followed by integration and interface testing in preparation for the final defense in December.

5. Challenges, technological and algorithmic choices

Image preprocessing and color removal (Nina Justo):

This module is responsible for the first stage of the OCR pipeline: loading an image, cleaning it, and preparing it for segmentation and recognition. It was implemented in C using the SDL2 and SDL2-image libraries, which provide efficient image manipulation and pixel-level access.

The program begins by initializing SDL and SDL-image to handle standard image formats such as PNG. Proper initialization is essential since most systems only support BMP loading by default with SDL. Once the initialization succeeds, the program loads an image given by the user as a command-line ar-

gument. During development, one of the main difficulties was linking SDL2 correctly across different environments. While it worked easily on Linux through package managers, Windows setups (via WSL or MinGW) required additional configuration and library paths.

After loading the image, the preprocessing step starts with a grayscale conversion implemented in the function called `grayscale`. This function iterates over every pixel of the image and replaces the red, green, and blue values with a single gray value computed as a weighted average of the three color channels. The chosen formula gives more importance to green and red components to better represent how human vision perceives brightness. Early versions of this function had small bugs due to different pixel formats and alignment issues between images, but converting the surface to a consistent RGB format solved those problems.

The second step is contrast enhancement, handled by the function `linear-contrast`. It analyzes the brightness of every pixel to find the minimum and maximum values, then stretches all values so that the darkest pixel becomes 0 and the brightest becomes 255. This simple normalization increases clarity and makes the letters in the grid stand out more clearly. During implementation, the developer initially noticed that flat images, where all pixels had the same brightness, caused a division by zero, so an additional safety condition was added.

Finally, the module allows manual image rotation. After saving the grayscale and contrast-enhanced versions, the program asks the user to input a rotation angle. The rotation is handled by the function `rotate-surface`, which computes the new dimensions of the rotated image and applies a geometric transformation to map each destination pixel back to a position in the source image. This operation uses basic trigonometric transformations with cosine and sine to correctly reposition pixels. Out-of-bounds pixels are filled with black, avoiding artifacts on the borders. The main challenge here was ensuring that the new image dimensions were large enough to fit the entire rotated picture; otherwise, parts of the image could get cropped.

The rotation step was especially useful for handling cases where the photographed puzzle was not perfectly aligned. By allowing manual rotation,

the user can visually correct such imperfections before the segmentation phase. While the rotation currently uses a simple nearest-neighbor interpolation, which can cause a slightly rough appearance, it was sufficient for the project's needs.

Overall, this part of the project successfully handles image loading, preprocessing, and rotation using only SDL2 and standard C libraries. The resulting images are stored as BMP files for later stages of the OCR pipeline. This preprocessing ensures that the neural network and solver modules receive clean, properly aligned images, greatly improving the accuracy of the final OCR results.

Neural network for OCR (Helios Bringuet):

This code implements a simple feedforward neural network trained to learn the XNOR logical function. The program is written entirely in C, using only the standard libraries for mathematical operations, random initialization, and timing. It serves as a small-scale demonstration of how neural networks can learn non-linear relationships through gradient-based training, and it also forms the conceptual base for the more complex OCR neural model used in the full project.

The XNOR function is particularly interesting because it is not linearly separable, meaning that a single-layer perceptron cannot learn it. To handle this, the network defined here includes a hidden layer with two neurons. The structure is minimal but sufficient to represent non-linear combinations of the two binary inputs. The network uses a classic sigmoid activation function, which maps any real value to a range between 0 and 1. This choice simplifies both forward propagation and derivative calculation since the sigmoid function is smooth and differentiable.

The program starts with the definition of the activation functions. The sigmoid and its derivative are implemented separately for clarity. During development, one of the challenges was ensuring numerical stability. The exponential function can easily overflow if large negative or positive values are passed to it. Although this network uses small random initial weights, experiments showed that using larger initialization ranges caused outputs to

saturate near 0 or 1, making learning impossible. Therefore, the initialization was limited to random values between -0.5 and 0.5 to maintain proper activation behavior.

The core of the network is represented by the structure named XNORNetwork. It stores six weights and three biases: four weights for the connections between the input and hidden layer, two for the connections between the hidden layer and output, and one bias per neuron. Keeping the network in such a compact format made debugging easier, and it clearly shows the mapping from theoretical concepts to actual code.

The function `init-network` assigns random initial values to each weight and bias. This randomness ensures that neurons start with different parameters and learn diverse features. If all weights were initialized to the same value, the network would remain symmetrical and fail to learn. Random seeding with the current time ensures that each execution begins with a different configuration, producing slightly different convergence behaviors.

The training process is implemented in the function `train`. It uses the standard backpropagation algorithm combined with gradient descent to adjust weights. For each of the four possible input pairs (0,0), (0,1), (1,0), and (1,1), the network computes a prediction through a forward pass, compares it to the expected XNOR output, and then updates all weights to minimize the difference. The learning rate `eta` controls how large the updates are at each iteration. The code prints the total error every 1000 epochs, allowing the user to observe the convergence process. Early testing showed that smaller learning rates made convergence very slow, while higher ones caused oscillations. The chosen value of 0.5 offered a good compromise.

The feedforward function calculates the network output by propagating the input through the two hidden neurons and then to the output neuron. Each neuron applies the weighted sum of its inputs, adds its bias, and passes the result through the sigmoid function. This modular design makes it easier to understand the internal transformations during debugging.

After training, the test function evaluates the network on all possible input pairs and prints the results. If training was successful, the outputs for (0,0)

and (1,1) should be close to 1, while (0,1) and (1,0) should be close to 0. In practice, the results are not perfectly exact because the sigmoid activation limits the precision to approximate values around 0 and 1.

During development, one of the main difficulties was implementing the back-propagation equations correctly. Small sign or derivative mistakes could lead to the network diverging or producing no improvement over time. To debug this, intermediate values such as deltas and gradients were printed during early tests, confirming that weight updates moved in the expected direction. Another challenge was ensuring that the network generalized properly from random initialization rather than memorizing input-output pairs. Although this network is extremely small, it provides a solid demonstration of how a neural system learns through repeated exposure and error correction.

Overall, this code provides a compact yet functional example of neural learning implemented from scratch. It demonstrates the essential components of modern machine learning — feedforward computation, activation functions, error calculation, and gradient-based parameter updates — without relying on external frameworks. This basic understanding of neural computation later served as the foundation for implementing larger networks used in the OCR solver, where similar mathematical principles are applied to recognize handwritten or printed characters from processed image data.

Grid and word list detection (Martin Doillon):

This program is one of the most complex parts of the preprocessing stage of the project. It is a standalone C program designed to automatically detect and separate the two main regions of a word-search puzzle image: the grid of letters and the word list. The code relies heavily on the SDL2 and SDL2-image libraries for image loading, pixel manipulation, and saving intermediate processing results. It takes as input an image file, typically in PNG or JPG format, and produces two BMP images as output: one binary thresholded version of the image (stage-bin.bmp) and one annotated layout image (stage-layout.bmp) where the detected grid and word list are outlined in red and blue rectangles. Additionally, it outputs two small text files containing the coordinates of these detected regions, which can later be used by the segmentation or OCR modules.

At the beginning of the program, a helper function called `compute-otsu-threshold` implements Otsu’s method, a well-known global thresholding algorithm in image processing. This function calculates a grayscale histogram of the image and determines an intensity threshold that maximizes the variance between foreground and background classes. This algorithm allows the program to automatically adapt to different lighting and contrast conditions without requiring manual adjustment. During development, one challenge faced was ensuring numerical stability and avoiding overflow when dealing with large pixel counts. To solve this, the developer used 64-bit accumulations and double precision for variance calculations.

In the main function, the program first checks command-line arguments and initializes SDL2 and SDL2-image. If initialization fails, it displays detailed error messages to help diagnose missing dependencies. The image is then loaded and converted to a 32-bit ARGB format to simplify pixel access. At this point, two new surfaces are prepared: one for the grayscale binary output and one for colored visualization. The code then locks the pixel data for direct access. This approach avoids high-level abstractions and works efficiently at the byte level, which was chosen for performance and full control over pixel processing.

The next step computes the grayscale intensity of each pixel using the standard luminosity formula, which gives more weight to green and less to blue. Each grayscale value contributes to a histogram used by the Otsu function to compute the optimal threshold. Once the threshold is known, the program performs a binary conversion: pixels below the threshold become black (foreground), and others become white (background). During this pass, the program also builds column and row projections (col-sum and row-sum arrays) that count the number of dark pixels per row and column. These projections are key to detecting large blank areas, which typically correspond to the separation between the grid and the word list.

Another difficulty encountered during implementation was handling images with different orientations or spacing. Some puzzle images had very thin or irregular gaps between the grid and the word list, which made threshold-based segmentation unreliable. To handle these cases, the developer introduced a

robust gap-detection algorithm. It scans the row and column projections to find the longest continuous run of empty space (where no black pixels are found). Depending on whether the best gap is horizontal or vertical, the program decides whether the grid and word list are stacked vertically or placed side-by-side.

Once a potential separation is detected, the program divides the image into two clusters. Each cluster is analyzed by counting black pixels to determine which one likely corresponds to the grid and which one to the word list. The cluster with the highest density of black pixels is assumed to be the grid, since it usually contains a dense matrix of letters, while the other, sparser region is assumed to be the word list. This heuristic was found to work well across multiple test cases, though during early experiments it sometimes failed on images with thick borders or uneven lighting. To address this, small corrections were applied, such as excluding completely white rows and columns before computing the histograms.

After segmentation, the program draws red and blue rectangles on the original image surface to mark the detected regions. This visual output helps verify the accuracy of the algorithm. The grid is drawn in red, and the word list is drawn in blue. These colors were chosen for contrast against typical black-and-white puzzle images. The result is then saved as `stage-layout.bmp`. The binary thresholded version is saved separately as `stage-bin.bmp`, which can be used for letter segmentation and OCR training. The region coordinates are also saved into text files `grid-roi.txt` and `wordlist-roi.txt`, allowing later modules to process only the relevant areas.

Overall, this code represents an important milestone in the project. It demonstrates an effective combination of classical image processing techniques, direct pixel manipulation with SDL2, and adaptive thresholding via Otsu's method. The implementation was not straightforward: several issues appeared with image formats, pitch alignment, and memory access. For instance, the team discovered that some SDL surfaces had unexpected padding bytes per row, which caused misaligned pixels when performing manual loops. Locking and unlocking surfaces properly solved these issues. Another recurring problem was that BMP saving sometimes produced inverted colors due

to format mismatches, which was fixed by ensuring consistent ARGB8888 formats across all surfaces. Despite its complexity, the final version of the program works reliably and serves as a foundation for later stages, including letter segmentation and optical character recognition.

Solver algorithm (Emile Lassalle):

This C program is a compact yet robust implementation of a word search solver that operates on a grid of letters stored in a text file. It reads the grid, validates its format, converts its contents to uppercase for consistency, and then searches for a given word across all possible directions in the grid. The program follows a procedural structure typical of low-level C design and emphasizes safety, correctness, and memory hygiene through careful error handling and explicit cleanup routines.

The logic of the program is organized into two main components: grid parsing and word searching. The grid parsing process, implemented in the function `read-grid`, reads the contents of a text file line by line and constructs a dynamically allocated 2D array representing the grid. Each line corresponds to one row in the grid, and the program ensures that all rows have equal length, meaning the grid must be rectangular. The function uses `getline` to read lines dynamically, adapting to variable input lengths without risking buffer overflows. It also performs strict validation: every character must be alphabetic and uppercase. Lowercase letters are automatically converted to uppercase to ensure case-insensitive matching, while spaces or special characters trigger an error.

Error handling is particularly strong in this part of the program. Whenever an inconsistency or invalid input is detected—such as lines of different lengths, invalid characters, or memory allocation failures—the function releases all allocated memory and returns an appropriate error code. This makes the code resilient and prevents memory leaks, which are common pitfalls in C when dealing with dynamic memory. Moreover, the function uses a double-pointer grid representation (`a char**`), allowing for flexible allocation and straightforward access to each cell. Once the grid is successfully built, the number of rows and columns is returned through output parameters, giving the rest of the program an accurate view of the data structure.

After the grid is loaded, the program proceeds to the search stage. The search-word function is responsible for scanning the entire grid in search of the target word. It uses a brute-force approach by iterating through every position in the grid and checking whether the word begins at that point. For each potential starting position, the function tests all eight possible directions: left, right, up, down, and the four diagonals. These directions are encoded as small integer vectors, which makes the search loop concise and systematic. For each direction, the function advances step by step, comparing the letters of the word to the corresponding characters in the grid. If a mismatch occurs or the path goes out of bounds, the search for that direction stops immediately. If all letters of the word are matched successfully, the program records the start and end coordinates of the word using two Point structures and returns success.

The helper functions used throughout the code reinforce modularity and readability. The uppercase function converts a string to uppercase in place, ensuring consistent comparisons, while in-bounds provides a simple and reliable way to check whether a coordinate lies within the valid limits of the grid. These small utilities reduce code duplication and make the logic easier to maintain.

The main function orchestrates the execution sequence and manages user interaction. It begins by checking that exactly two command-line arguments are provided: the path to the grid file and the word to search. It then normalizes the search word by converting it to uppercase and calls read-grid to load and validate the grid. If the grid fails to load correctly, the program exits gracefully, having released all resources. Otherwise, it calls search-word to locate the word in the grid. If the word is found, the program prints its coordinates in the format (x1,y1)(x2,y2); if not, it simply outputs "Not found". Afterward, all dynamically allocated memory for the grid is freed, ensuring clean termination.

The overall design is both simple and careful. The code avoids the use of global variables and passes all data through function arguments, which enhances modularity and makes it easier to integrate or reuse individual components. It also handles potential edge cases, such as empty files, uneven lines,

or invalid input symbols, demonstrating a defensive programming mindset. While the algorithm is not optimized for very large grids—it performs a linear scan for every position and direction—it guarantees correct results and full coverage of all search directions. The computational complexity is proportional to the number of grid cells multiplied by the word length and the number of directions, which is acceptable for typical puzzle sizes.

In conclusion, this program is an efficient and reliable C implementation of a word search solver. It demonstrates strong attention to detail in file parsing, input normalization, and memory management, all while maintaining a clear and logical structure. The code reflects a disciplined approach to low-level software design: every allocation is checked, every resource is freed, and every error condition is handled. It provides a solid foundation for more advanced developments, such as integrating graphical visualization or automated grid extraction from images, while remaining fully functional as a standalone console tool for solving word grids.

6. Conclusion

In conclusion, as we approach our first defense, the project has made significant progress in terms of individual module development, even though full integration has not yet been completed. Each team member has successfully implemented their assigned components: image preprocessing with color removal and manual rotation, neural network-based character recognition, segmentation of grids and word lists, and the solver algorithm capable of handling the reconstructed grids. These elements provide a solid foundation for the final application, demonstrating that the core functionalities are operational in isolation.

The team has also established a robust workflow using Git, with dedicated branches for each module, allowing parallel development while maintaining version control and reducing conflicts. Weekly in-person meetings at EPITA have been instrumental in aligning technical decisions, resolving issues, and ensuring that each module meets the expected specifications. While the

current state of the project does not yet allow for complete end-to-end execution, the progress achieved so far positions the team well for the upcoming integration phase.

For the next stages, the focus will be on combining these components into a cohesive system, debugging interactions between modules, and ensuring that the application can process images from start to finish reliably. Early tests suggest that once integration is complete, the system will be able to load, preprocess, recognize, and solve word search puzzles as intended. Overall, the work accomplished up to this point demonstrates a clear trajectory toward a fully functional OCR Word Search Solver, and the first defense will provide an opportunity to showcase the individual modules, discuss implementation strategies, and outline the plan for final integration and refinement.